

DukanBook

AI Voice + Text Assistant for Shopkeepers

Technical Documentation

Prepared for internal understanding of the system: architecture, technology stack, modules, data model and how each part connects.

Project: NestDukanBook Internship • Authors: Amaan Khan & Srishti Rathi

1. Introduction

DukanBook is a digital *bahi-khata* (ledger) for small Indian shopkeepers, with a voice and text AI assistant built on top of it. A shopkeeper can talk to it in Hindi, English or a mix (Hinglish) to keep accounts, set payment and call reminders, and get reliable answers to everyday business questions such as GST and loans. The guiding idea is that keeping accounts should feel as easy as talking to a *munshi*, with no complicated software to learn.

The system is deliberately split into independent modules: a structured ledger, the conversational assistant, a reminders module and a knowledge engine. Each part is simple to use and test on its own, and all of them read and write the same database, so the data stays consistent across the whole app.

2. System Architecture

The application follows a three-layer design. The frontend talks only to the backend over HTTP; the backend holds all the logic and is the only layer that touches the database. This keeps a single source of truth and lets us swap or add frontends without changing the core.

Layer	Responsibility	Built with
Presentation (UI)	Screens the shopkeeper sees and interacts with. Two frontends share the same backend: the original Streamlit app and the new branded web app.	HTML/CSS/JavaScript; Streamlit
Application (API + logic)	Request handling, the AI brain and tool-calling, the ledger rules, reminders, the knowledge engine and voice.	FastAPI (Python) + Uvicorn
Data	Persistent storage of parties, transactions, reminders and knowledge embeddings.	SQLite

Request flow (example):

User speaks or types > Frontend sends it to the FastAPI backend > The AI brain interprets the request and calls the right tool (record a transaction, create a reminder, answer from the knowledge base, etc.) > The tool reads/writes SQLite > The backend returns a reply > The frontend shows it (and, for voice, plays it back).

3. Technology Stack

Everything runs on free, open tools. The assistant uses a local AI model, so no paid API key is required for it to work.

Area	Technology	Role in the project
Backend framework	FastAPI + Uvicorn	REST API, request validation, serving the web app.
Language	Python 3	All backend logic and tests.
Database	SQLite	Single-file store for parties, transactions, reminders, knowledge.
Local LLM	Ollama (OpenAI-compatible)	The 'smart' assistant; runs on the machine, no key.
LLM fallbacks	Gemini / Groq (optional)	Cloud models if a key is provided instead of Ollama.
Embeddings (RAG)	fastembed (multilingual-e5-large)	Turns knowledge notes and questions into vectors.
Vector math	NumPy	Cosine-similarity search over stored embeddings.
Speech to text	faster-whisper (offline Whisper)	Transcribes Hindi/English/Hinglish speech, no key.
Text to speech	edge-tts (neural Indian voices)	Speaks the reply back, no key.
Web frontend	HTML, CSS, vanilla JavaScript	The branded replica of the product app.

Rapid frontend	Streamlit	The original multi-page app used during development.
Config	python-dotenv	Reads settings from a .env file.
Testing	pytest	Automated tests across all layers.

4. Project Structure

Path	What it contains
app/main.py	FastAPI app: all API endpoints, and it serves the web app at /app.
app/brain.py	The assistant's controller: decides how to answer, runs the LLM tool loop.
app/llm.py	Talks to the local Ollama model (or Gemini/Groq); defines the tools.
app/parser.py	Rule-based offline fallback that works with no LLM.
app/tools.py	Ledger/reminder actions the AI can call; phone, WhatsApp and call links.
app/rag.py	Knowledge engine: embeds notes, retrieves the most relevant one.
app/voice.py	Speech to text (faster-whisper) and text to speech (edge-tts).
app/db.py	SQLite schema, migrations and all read/write helpers.
app/config.py	Loads .env and reports whether the LLM / voice are available.
data/knowledge/	The reference notes (GST, income tax, MSME, MUDRA, FSSAI, UPI, etc.).
web/	The new HTML/CSS/JS web app (index.html, styles.css, app.js).
ui/	The Streamlit app (dashboard, Khata, AI Assistant, Reminders).
tests/	The pytest suite covering db, tools, parser, brain, llm and the API.

5. Backend API (FastAPI)

The backend exposes two clearly separated surfaces over the same database: the structured ledger and the AI assistant. Reminders and health round it out.

Method & Path	Purpose
GET /health	Reports status and whether the LLM and voice are available.
POST /chat	Text message to the assistant; returns the reply.
POST /voice/chat	Audio in; transcribes, answers, returns text + spoken audio.
GET /parties	List all customers and suppliers with balance and phone.
POST /parties	Create a customer or supplier account.
POST /parties/{id}/phone	Add or update a party's phone number.
GET /parties/{id}	One party with balance and full transaction history.
POST /transactions	Record a credit (udhaar) or debit (payment).
GET /reminders	List reminders with ready WhatsApp and call links.
POST /reminders	Create a call or WhatsApp reminder.
POST /reminders/{id}/done	Mark a reminder as completed.
/app (static)	Serves the branded HTML/CSS/JS web app, same origin as the API.

6. Database (SQLite)

The data lives in a single SQLite file (`dukanbook.db`). Four tables model the whole system. The schema is created on startup, and a small migration step can add new columns to an existing database without losing data.

Table	Key columns	Purpose
party	id, name, type (customer/supplier), phone, created_at	The shopkeeper's customers and suppliers.
transaction	id, party_id, type (credit/debit), amount, note, txn_date, source	Every udhaar given or payment received.
reminder	id, party_id, due_at, amount, channel (call/whatsapp), phone, status	Call and payment reminders / call requests.
kb_chunk	id, text, source, embedding (BLOB)	Knowledge notes plus their vector embeddings for retrieval.

Balance convention

A party's balance is never stored; it is always computed as **sum(credit) - sum(debit)**. A positive balance means the party still owes the shopkeeper (you will get); a negative balance means the shopkeeper owes the party (you will give). Because the balance is derived, it can never drift out of sync with the transactions.

7. The AI Brain and Tool-Calling

When a message arrives, the brain (`app/brain.py`) decides how to handle it:

- Simple greetings and small talk get a quick, friendly reply.
- Deterministic commands (for example setting a phone number) are handled directly so they work even without the LLM.
- Otherwise the request goes to the language model, which is given a set of **tools** it can call: record a transaction, create an account, set a reminder, look up a balance, search the knowledge base, and so on.
- The model picks the right tool and arguments; the backend runs the tool against the database and returns the result, which the model turns into a natural reply.
- If no LLM key is set, a rule-based parser handles the common cases so the app still works.

This tool-calling design keeps the AI grounded in real actions and real data rather than free-form text, which makes it reliable for money-related tasks.

8. The Local Language Model (Ollama)

The assistant's intelligence runs locally through Ollama, which exposes an OpenAI-compatible endpoint on the machine. This means no cloud key, no per-message cost and no usage limits.

The provider is chosen automatically by priority, so any available option works. All three speak the same OpenAI-compatible format, so only the endpoint, model and auth header change.

Priority	Provider	When it is used / model
1 (default)	Local Ollama	When OLLAMA_MODEL is set (e.g. qwen3:8b). No key, no billing, always on.
2	Gemini	When GEMINI_API_KEY is set. Endpoint: Google generativelanguage; model gemini-2.0-flash.
3	Groq	When GROQ_API_KEY is set. Endpoint: api.groq.com; model llama-3.3-70b-versatile.
Fallback	Local Ollama	If none is configured, it still tries a local model; else the offline parser answers.

Some local models add <think> reasoning tags to their output; these are stripped before the reply is shown. If a cloud call ever fails, the assistant falls back to the offline parser so the app never breaks.

9. Offline Fallback Parser

So the app is never fully dependent on a model, a lightweight rule-based parser (`app/parser.py`) understands the most common commands: recording udhaar and payments, creating accounts, setting a phone number and creating reminders. It also extracts dates and times from natural phrasing and is careful with edge cases (for example, not confusing '10 baje' with an amount of 10 rupees).

10. Knowledge Engine (RAG)

Business questions are answered using Retrieval-Augmented Generation, so answers are grounded in prepared reference material instead of being guessed.

Pipeline

- The reference notes in `data/knowledge/` are split into chunks.
- Each chunk is converted into a vector using the multilingual-e5-large embedding model (via `fastembed`), which works well for Hindi, English and Hinglish.
- The vectors are stored as BLOBs in the `kb_chunk` table.
- At question time, the question is embedded the same way and compared to every stored chunk using cosine similarity.
- The most relevant chunk(s) are passed to the assistant, which answers from that material.

Sources cover GST, income tax, MSME / Udyam registration, MUDRA loans, the FSSAI licence, UPI limits and PM SVANidhi, prepared from official portals. Retrieval accuracy is checked with a small evaluation script.

11. Voice (Speech In and Out)

Direction	Library	Detail
Speech to text	faster-whisper	An offline build of Whisper; transcribes speech and detects the language. No key.
Text to speech	edge-tts	Neural Indian voices (for example Hindi 'Swara', English 'Neerja'). No signup key, needs internet.

In the web app, the microphone records audio, sends it to `/voice/chat`, and the reply is both shown as text and played back as speech.

12. Frontend

Two frontends share the one backend and database:

New web app (web/)

A branded single-page app in HTML, CSS and vanilla JavaScript, made to match the real `nestdukanbook.com` product: a teal brand bar with the DukanBook wordmark, Customers / Suppliers tabs, the 'you will get / you will give' summary, search and add, an account detail screen with history, phone with call and WhatsApp buttons, a reminders screen, and the AI Assistant (text + voice) in the centre of the bottom navigation. It is served by FastAPI at `/app`, so it talks to the API on the same origin and stores everything in the shared database.

Streamlit app (ui/)

The original multi-page app (Dashboard, Khata, AI Assistant, Reminders) used during development. It calls the same API, so both frontends stay consistent.

13. Testing

The project is developed test-first. The `pytest` suite covers the database layer, the ledger/reminder tools, the offline parser, the AI brain and the LLM tool schema, plus the API endpoints. New behaviour is added with its tests, and the full suite is run before any feature is considered done.

14. Configuration and Running

Install dependencies once, then start the app:

```
pip install -r requirements.txt
python run.py
```

This starts the backend and the screens together. The web app is available at `http://localhost:8000/app/`. Voice runs on local libraries (no key). To use the local LLM, run Ollama and set `OLLAMA_MODEL` in a `.env` file.

15. Data Handling and Scaling Plan

- **Single source of truth:** all data goes through the backend API into one SQLite database, so both frontends and the assistant always agree.
- **New sections:** upcoming Stock and Bills screens get their own tables (for example product, bill) and a few endpoints, following the same pattern; existing features are untouched.
- **Multi-shop:** when each shopkeeper needs their own data, a shop/user table is added and every row is tagged with a shop id, which is easy because all access goes through one data layer.
- **Hosting:** SQLite is ideal for development and a single user. For a hosted product with many users, the store moves to PostgreSQL; because the app only talks to the data layer, this changes one area, not the whole codebase.
- **Backups:** today a backup is simply copying the single database file.

This document reflects the working setup: a local Ollama model, an expanding knowledge base, and a branded web app served by the same FastAPI backend that powers the assistant.